Đại học quốc gia Thành phố Hồ Chí Minh
Trường đại học công nghệ thông tin

khoa khoa học máy tính

môn học: cấu trúc dữ liệu và giải thuật nâng cao - CS523.Q21

Bài tập 2
Tạo ứng dụng minh họa sắp xếp ngoại

Sinh viên thực hiện:

Nguyễn Thiên Bảo – 23520127

Giảng viên hướng dẫn:

ThS.Nguyễn Thanh Sơn

Mục lục

1 Giới thiệu tổng quan **2**

2 Thuật toán **2**

3 Hướng dẫn sử dụng **3**

3.1 Bước 1: Khởi động và Nạp dữ liệu 3

3.2 Bước 2: Thiết lập mô hình Bộ nhớ đệm (Page Model) 4

3.3 Bước 3: Chạy thuật toán 5

3.4 Bước 4: Quan sát trực quan hóa (Visualization) 6

4 Github, Demo, Cài đặt và Khởi chạy **9**

4.1 Link mã nguồn & tài liệu liên quan 9

4.2 Video Demo 9

4.3 Yêu cầu hệ thống 10

4.4 Cài đặt các thư viện phụ thuộc 10

4.5 Khởi chạy và Đóng gói ứng dụng 10

1 Giới thiệu tổng quan

Ứng dụng "External Sort Visualizer"là một phần mềm Desktop được thiết kế để trực quan hóa cách thức hoạt động của thuật toán Sắp xếp trộn ngoài (External Merge Sort). Chương trình hỗ trợ người dùng nạp các tập tin nhị phân (.bin) chứa dữ liệu là các số thực 8-bytes (double-precision floating-point).

Đặc điểm nổi bật của ứng dụng là mô phỏng chính xác mô hình quản lý bộ nhớ IO-aware (page-based). Thay vì xử lý toàn bộ dữ liệu trên RAM cùng lúc, phần mềm minh họa cách máy tính chia nhỏ dữ liệu thành các trang (Pages), nạp vào bộ nhớ đệm (Buffer) có giới hạn, sắp xếp và ghi ngược lại đĩa cứng thông qua nhiều lượt quét (Passes).

2 Thuật toán

Ứng dụng minh họa 3 phương pháp tiếp cận cốt lõi của bài toán Sắp xếp ngoại:

- Mô hình I/O cơ bản: Toàn bộ dữ liệu được đọc/ghi theo từng trang (Page). Chi phí I/O của mỗi Pass quét qua toàn bộ dữ liệu luôn là $2N$ thao tác (trong đó N là tổng số trang của file, bao gồm N lần đọc và N lần ghi).

- 2-Way Merge Sort (Trộn 2 đường): Thuật toán sử dụng 3 trang bộ nhớ đệm (2 trang Input, 1 trang Output) để trộn từng cặp dãy dữ liệu (runs) đã sắp xếp. Thuật toán liên tục gộp 2 dãy thành 1 dãy lớn hơn cho đến khi chỉ còn một dãy duy nhất.

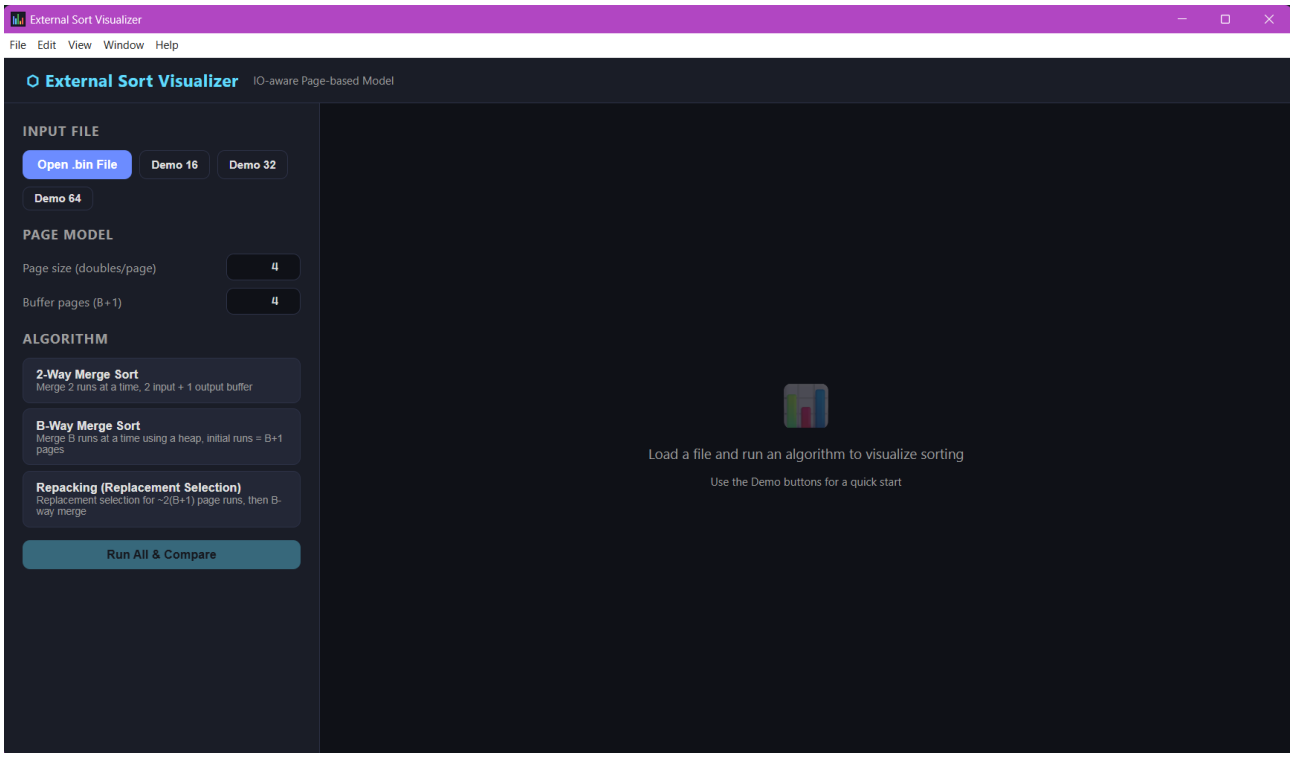
- B-Way Merge Sort (Trộn B đường): Mở rộng từ 2-Way, thuật toán sử dụng cấu trúc Min-Heap để trộn cùng lúc B dãy dữ liệu, tận dụng tối đa $B + 1$ trang bộ nhớ đệm. Các dãy ban đầu (Initial runs) được tạo ra với kích thước bằng đúng sức chứa của RAM (tức $B + 1$ trang).

- Repacking / Replacement Selection (Thay thế lựa chọn): Đây là kỹ thuật tối ưu hóa việc tạo dãy ban đầu. Bằng cách duy trì một Min-Heap và "đóng băng"(freeze) các phần tử mới đọc vào nếu chúng nhỏ hơn phần tử vừa được ghi ra Output, thuật toán có thể tạo ra các dãy ban đầu dài xấp xỉ gấp đôi dung lượng bộ nhớ đệm (trung bình $\sim 2(B + 1)$ trang). Việc này làm giảm số lượng dãy ban đầu, từ đó giảm số Pass cần trộn và tiết kiệm chi phí I/O.

3 Hướng dẫn sử dụng

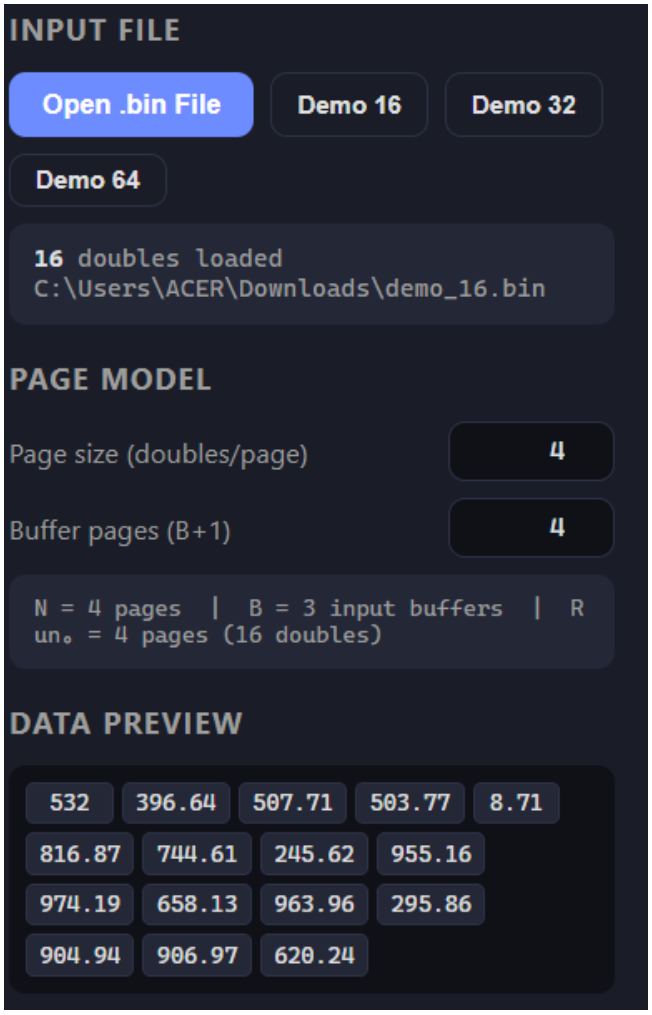
3.1 Bước 1: Khởi động và Nạp dữ liệu

Giao diện chính của chương trình tập trung các công cụ thiết lập ở thanh Sidebar (cạnh trái màn hình).



Hình 1: Giao diện ban đầu

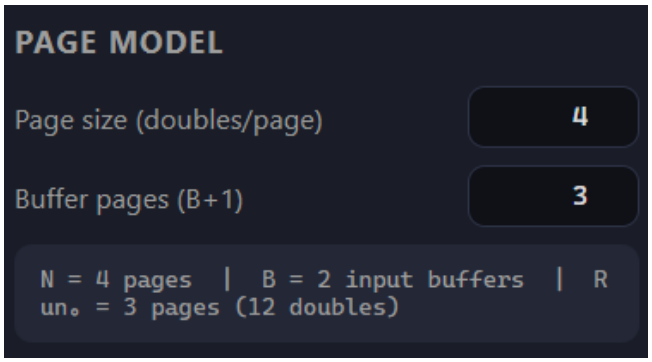
- Để sử dụng dữ liệu thực tế: Nhấn nút Open .bin File để chọn tập tin nhị phân từ máy tính.
- Để dùng thử nhanh: Nhấn các nút Demo 16, Demo 32, hoặc Demo 64. Phần mềm sẽ tự động sinh ra một file nhị phân chứa số lượng số thực tương ứng và nạp vào chương trình.
- Dữ liệu sau khi nạp sẽ hiển thị một phần trong khu vực DATA PREVIEW.



Hình 2: Chỗ để nạp và load file

3.2 Bước 2: Thiết lập mô hình Bộ nhớ đệm (Page Model)

Đây là bước quan trọng nhất để tạo ra giới hạn bộ nhớ ảo, giúp kích hoạt thuật toán sắp xếp ngoại.



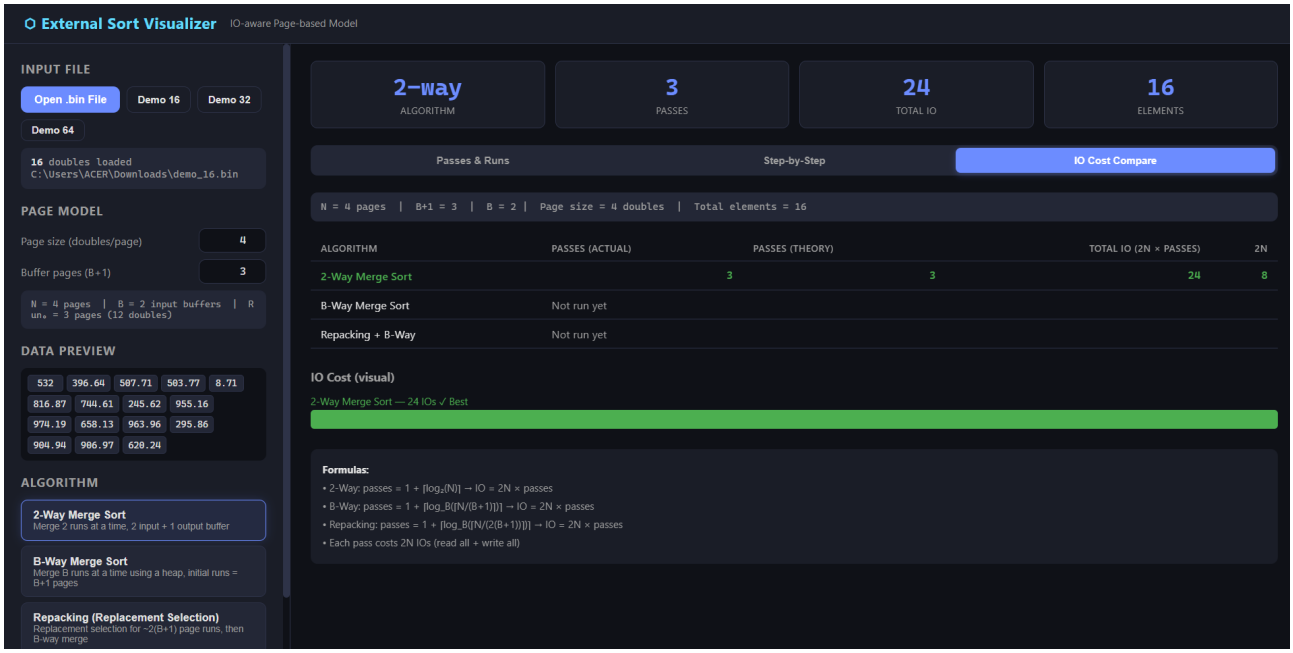
Hình 3: Chỗ thiết lập mô hình Bộ nhớ đệm

- Page size (doubles/page): Định nghĩa số lượng phần tử (số thực) mà một trang (Page) có thể chứa.

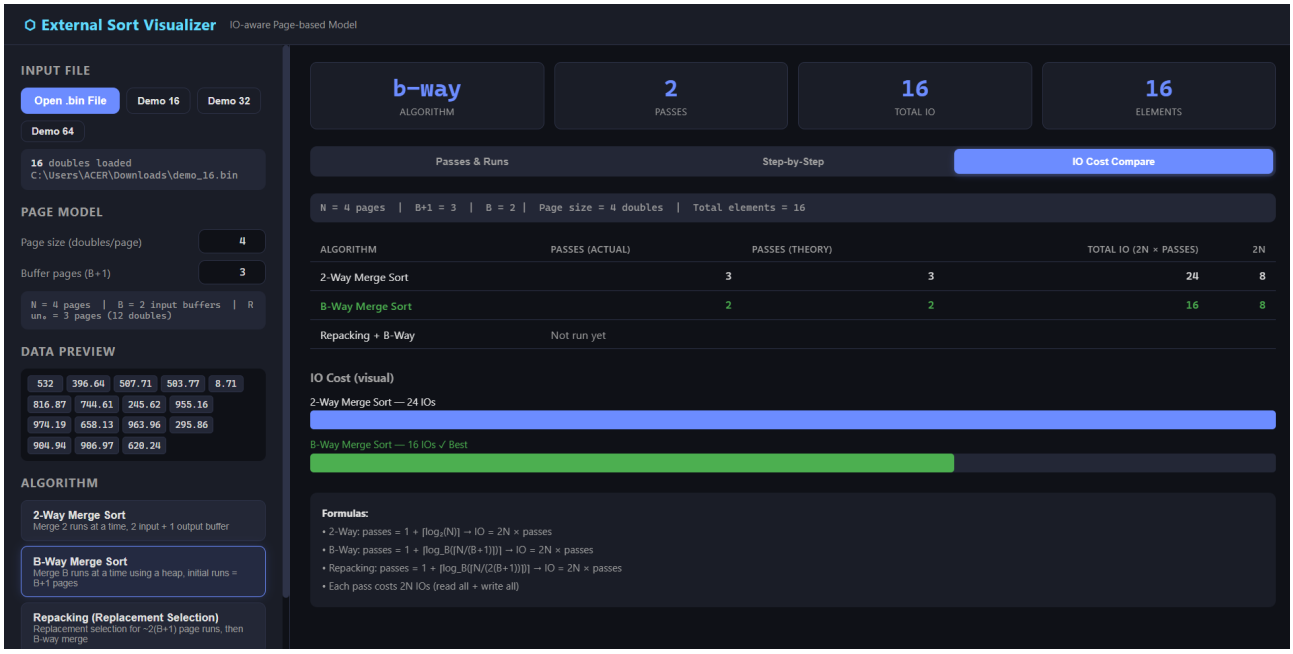
- Buffer pages (B+1): Cấu hình tổng số trang bộ nhớ đệm mà chương trình được phép sử dụng trong RAM.
- Chương trình sẽ tự động tính toán và hiển thị bên dưới các thông số: Tổng số trang của file (N), Số buffer đầu vào (B), và Kích thước dãy ban đầu được tạo ra.

3.3 Bước 3: Chạy thuật toán

Tại mục ALGORITHM, người dùng có thể click chọn 1 trong 3 thuật toán: 2-Way Merge Sort, B-Way Merge Sort, hoặc Repacking. App sẽ dựa vào số thuật toán đã bấm để tiến hành so sánh kết quả với nhau.

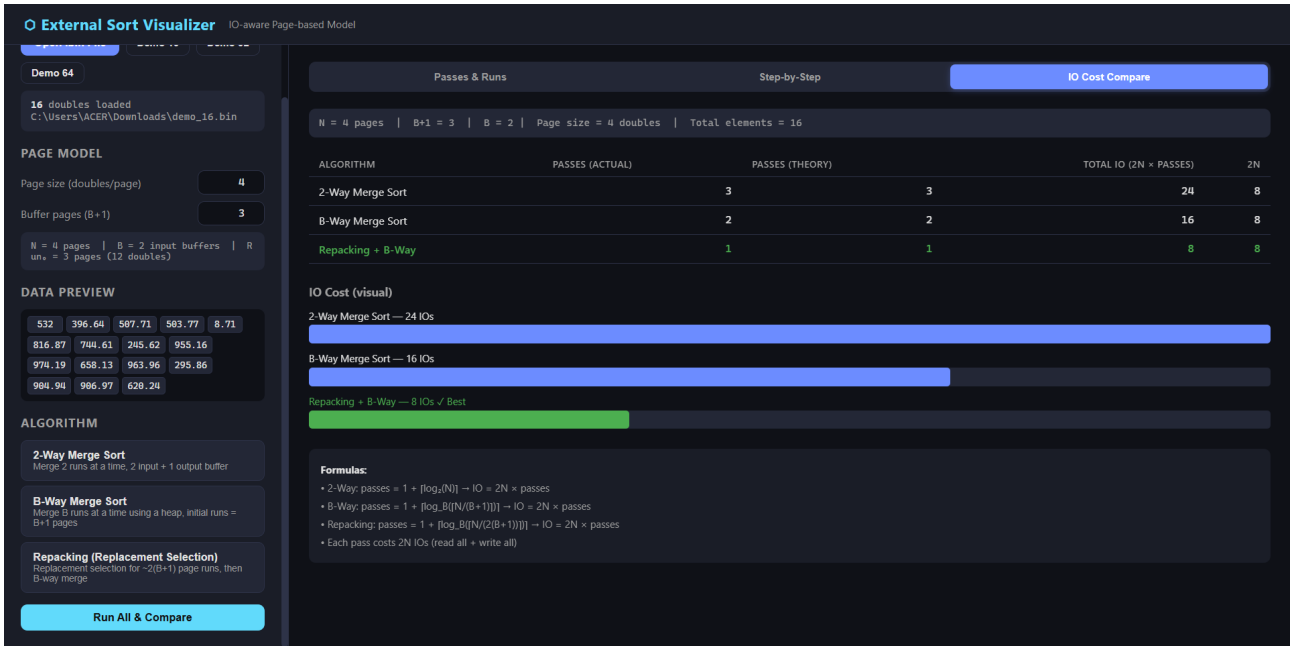


Hình 4: Kết quả khi bấm 1 thuật toán



Hình 5: Kết quả khi bấm thuật toán thứ 2

Hoặc, nhấn nút màu xanh Run All & Compare ở dưới cùng để hệ thống chạy song song cả 3 thuật toán trên cùng một tập dữ liệu.

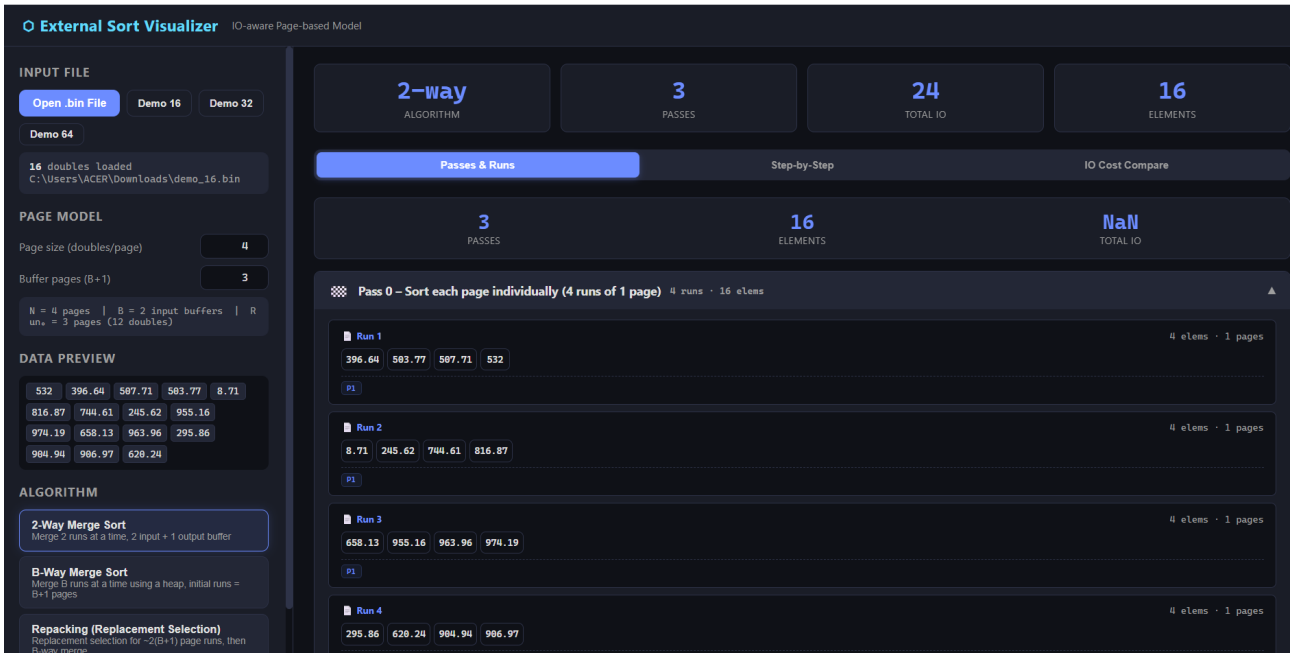


Hình 6: Kết quả khi bấm Run All & Compare

3.4 Bước 4: Quan sát trực quan hóa (Visualization)

Kết quả sẽ được hiển thị ở màn hình chính bên phải thông qua 3 Tabs:

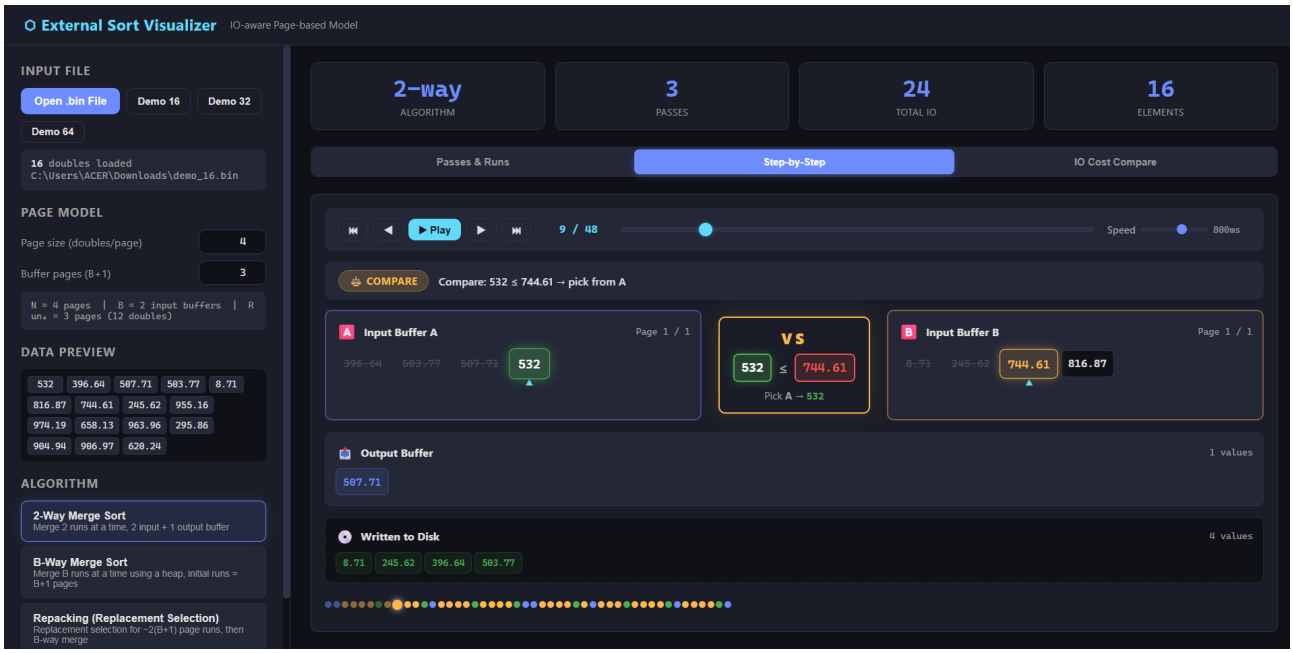
- Tab 1: Passes & Runs



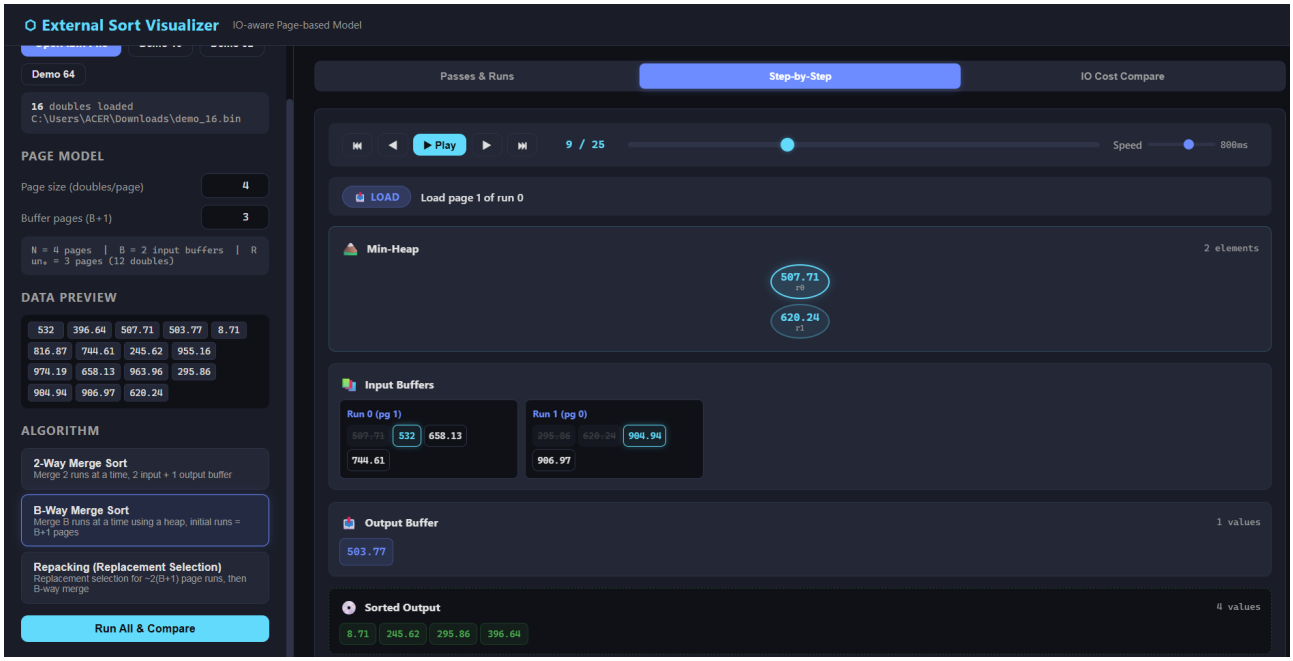
Hình 7: Tab 1: Passes & Runs khi chọn External Sort: 2-Way Merge Sort

- Cung cấp cái nhìn tổng quan về kết quả của từng lượt quét (Pass) của thuật toán đã chọn.
- Người dùng có thể mở rộng/thu gọn các thẻ (Cards) để xem chi tiết các dãy (runs) được tạo ra sau mỗi lượt trộn.
- Mỗi thẻ dãy sẽ hiển thị ranh giới từng trang (ví dụ: P1, P2, P3) để làm rõ tính chất IO-aware.

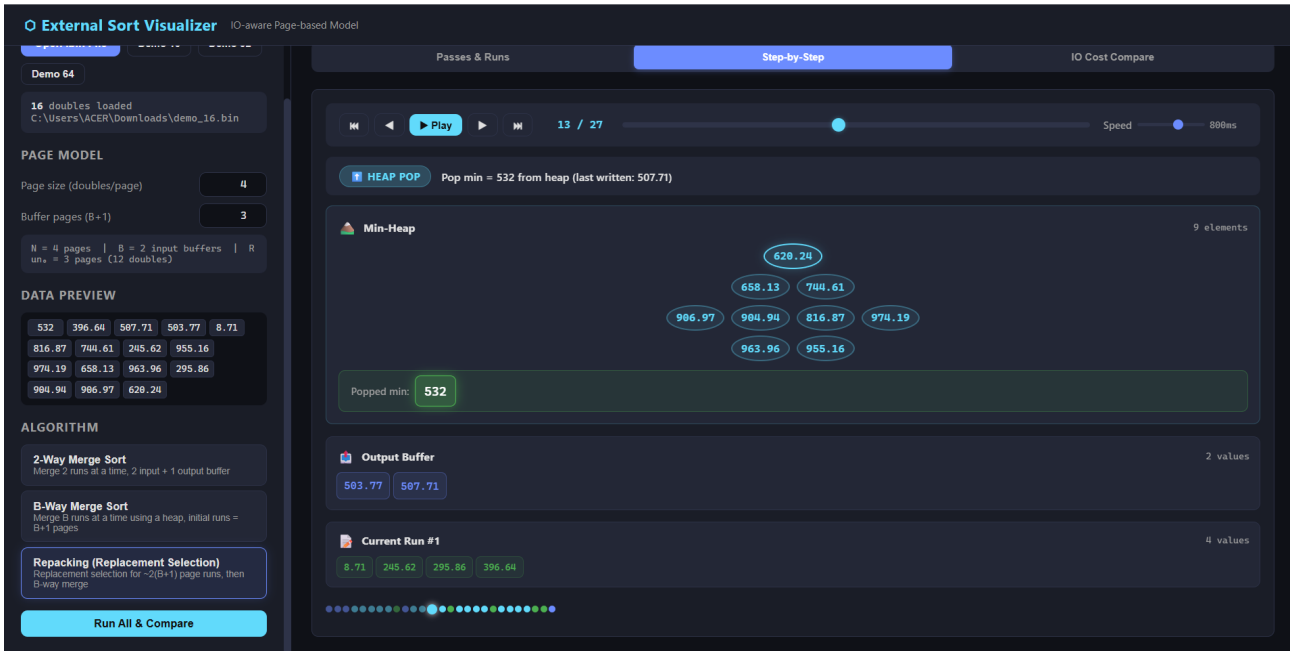
- Tab 2: Step-by-Step (Mô phỏng chi tiết)



Hình 8: Tab 2: Step-by-Step khi chọn External Sort: 2-Way Merge Sort



Hình 9: Tab 2: Step-by-Step khi chọn External Sort: B-Way Merge Sort

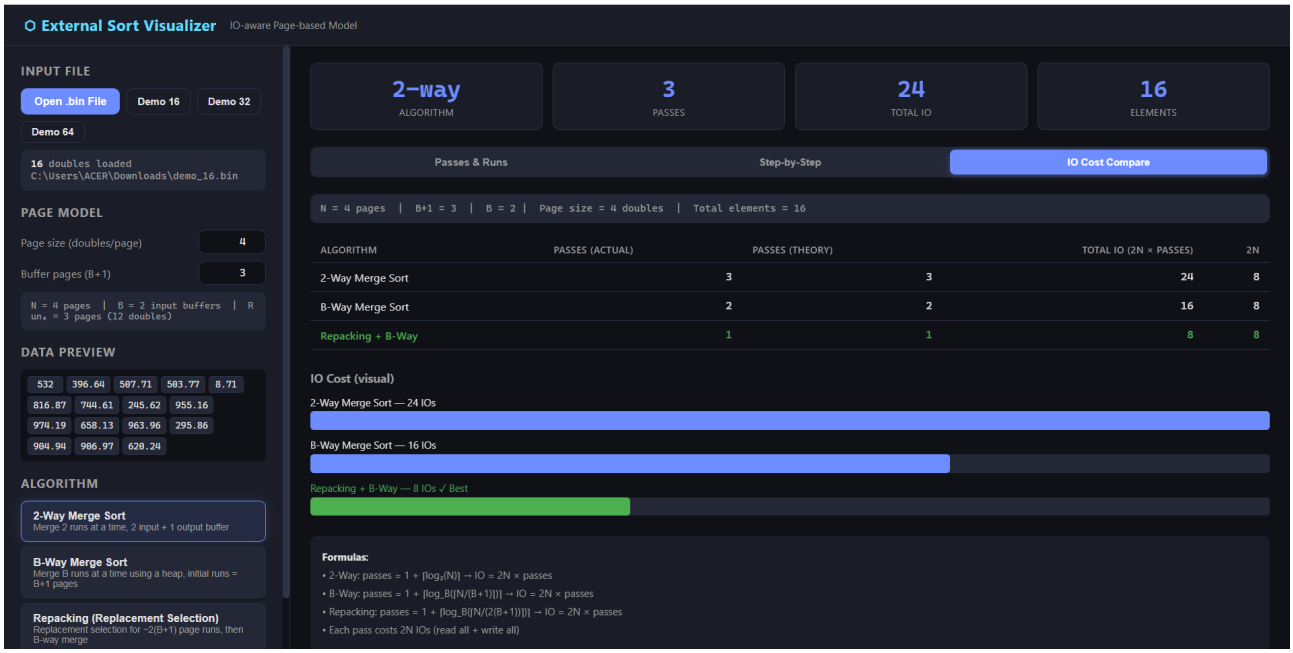


Hình 10: Tab 2: Step-by-Step khi chọn External Sort: Repacking (Replacement Selection)

- Đây là trình phát lại toàn bộ quá trình xử lý của thuật toán.
- Người dùng sử dụng thanh điều khiển (Nav bar) để Play/Pause, chỉnh tốc độ chạy, hoặc tua đến một bước bất kỳ (thông qua thanh trượt hoặc chuỗi dấu chấm timeline ở dưới cùng).
- Mỗi hành động của thuật toán (Tải trang vào Buffer, So sánh lấy Min, Ghi trang ra đĩa, Đóng băng phần tử) đều được hiển thị minh họa bằng màu sắc tương ứng. Mũi

tên chỉ mục (Pointer) sẽ di chuyển theo từng phần tử đang được xét trong Buffer.

- Tab 3: IO Cost Compare (So sánh hiệu năng)



Hình 11: Tab 3: IO Cost Compare

- Tab này trình bày một bảng đối chiếu chi tiết giữa 3 thuật toán về: Số Passes thực tế, Số Passes theo lý thuyết, và Tổng I/O tiêu tốn.
- Biểu đồ thanh ngang (Bar chart) sẽ làm nổi bật (màu xanh lá) thuật toán tối ưu nhất có chi phí I/O thấp nhất cho tập dữ liệu hiện tại.
- Khu vực dưới cùng ghi chú lại các công thức toán học tính toán độ phức tạp của từng thuật toán để tiện đối chiếu.

4 Github, Demo, Cài đặt và Khởi chạy

4.1 Link mã nguồn & tài liệu liên quan

Mã nguồn cũng như các tài liệu liên quan (README, Docstrings,...) đã có trên:

Github

4.2 Video Demo

Link Video Demo youtube

4.3 Yêu cầu hệ thống

- Node.js: Yêu cầu phiên bản 18 trở lên (ứng dụng đã được kiểm tra và hoạt động ổn định với phiên bản v22+).
- npm: Trình quản lý gói phần mềm (thường được cài đặt mặc định đi kèm với Node.js).

4.4 Cài đặt các thư viện phụ thuộc

Mở Terminal (hoặc Command Prompt / PowerShell), điều hướng (cd) vào thư mục gốc của dự án (nơi chứa file package.json) và chạy lệnh sau để tải về các gói thư viện cần thiết:

npm install

Lệnh này sẽ cài đặt toàn bộ các dependencies phục vụ cho dự án bao gồm React, Vite, Electron và các công cụ liên quan.

4.5 Khởi chạy và Đóng gói ứng dụng

- Chế độ Phát triển (Development mode):

  Để chạy ứng dụng phục vụ cho việc chỉnh sửa code và xem trước giao diện, chạy lệnh:

  npm run dev

  Lệnh này sẽ đồng thời khởi chạy máy chủ phát triển Vite (tại cổng 5173) và mở cửa sổ ứng dụng Electron. Chế độ này có hỗ trợ tính năng Hot-reload, giúp giao diện React tự động cập nhật ngay khi lưu code mới.

- Chế độ Đóng gói (Build Production):

  Để đóng gói ứng dụng thành một phần mềm độc lập chạy trên hệ điều hành Windows (file .exe), sử dụng lệnh:

  npm run dist

  Sau khi quá trình build hoàn tất, file cài đặt (.exe) sẽ được xuất ra và lưu trữ tự động trong thư mục dist_electron/.